

argument. You can also specify the keyword arguments in any order:

```
>>> formMolecule(hydrogen=2, oxygen=1)
'water'
>>> formMolecule(oxygen=1, hydrogen=2)
'water'
>>> formMolecule(carbon=8, hydrogen=10, nitrogen=4, oxygen=2)
'caffeine'
```

But you still have an unwieldy `def` statement with 118 parameter names. And what if new elements were discovered? You'd have to update the function's `def` statement along with any documentation of the function's parameters.

Instead, you can collect all the parameters and their arguments as key-value pairs in a dictionary using the `**` syntax for keyword arguments. Technically, you can name the `**` parameter anything, but it's usually named `kwargs` by convention:

```
>>> def formMolecules(**kwargs):
...     if len(kwargs) == 2 and kwargs['hydrogen'] == 2 and
```

```
kwargs['oxygen'] == 1:
```

```
...         return 'water'
...     # (rest of code for the function goes here)
...
>>> formMolecules(hydrogen=2, oxygen=1)
'water'
```

The `**` syntax indicates that the `kwargs` parameter can handle all keyword arguments passed in a function call. They'll be stored as key-value pairs in a dictionary assigned to the `kwargs` parameter. As new chemical elements are discovered, you'd need to update the function's code but not its `def` statement, because all keyword arguments are put into `kwargs`:

```
❶ >>> def formMolecules(**kwargs):
❷ ...     if len(kwargs) == 1 and kwargs.get('unobtainium') == 12:
...         return 'aether'
...     # (rest of code for the function goes here)
```